

C-CAT SECTION A — COMPUTER FUNDAMENTALS

MEGA REVISION GUIDE

17 Topics • All Conversions Explained • Exam MCQs • Memory Tricks

01 Number Systems
□ HIGH

02 Types of ROM
□ HIGH

03 Types of RAM
□ HIGH

04 Memory Hierarchy
□ HIGH

05 CPU & Components
□ HIGH

06 Algorithms & Flowcharts
□ HIGH

07 Register Memory
□ MEDIUM

08 Cache Memory
□ MEDIUM

09 IO Techniques
□ MEDIUM

10 Low vs High Level Language
□ MEDIUM

11 Evaluation of Languages
□ MEDIUM

12 OOP Overview
□ MEDIUM

13 Data Types & Structures
□ MEDIUM

14 Variable Declaration
□ MEDIUM

15 Flow Control Structures
□ MEDIUM

16 Debugging & IDEs
□ LOW

17 Functional Programming
□ LOW

All 17 topics covered in order of priority. Read chapters 1-6 first (HIGH), then 7-15 (MEDIUM), then 16-17 (LOW).

Computers work only in 0s and 1s. Number Systems define how we represent values in different bases and convert between them. This is the most fundamental and most-tested topic in every C-CAT paper.

1.1 THE FOUR NUMBER SYSTEMS

System	Base	Digits Used	Common Use
Binary	2	0, 1	CPU machine language, all internal processing
Octal	8	0-7	Unix permissions, older computing
Decimal	10	0-9	Everyday human numbers
Hexadecimal	16	0-9, A-F	Memory addresses, colour codes (#FF5733), error codes

Hex Letter Table — Memorise This Completely

A = 10 | B = 11 | C = 12 | D = 13 | E = 14 | F = 15

Hex uses letters because beyond 9, a single character must represent values 10 to 15.

1.2 UNDERSTANDING PLACE VALUE — The Foundation of All Conversions

Every number system uses position-based value. Each digit's actual value depends not just on the digit itself, but on WHICH POSITION it sits in. This is the most important concept before doing any conversion.

Decimal Example — Base 10:

Number: 3 4 5

Position: 2 1 0 (right to left, starting from 0)

Weight: 10^2 10^1 10^0 = 100 10 1

Value: $(3 \times 100) + (4 \times 10) + (5 \times 1) = 300 + 40 + 5 = 345$

Universal Rule: Position weight = (Base) raised to the power of that position number. Rightmost digit is always position 0. Moving left, the position number increases by 1 each step.

Powers of 2 — Memorise These 8 Values:

Position: 7 6 5 4 3 2 1 0

Value: 128 64 32 16 8 4 2 1

1.3 BINARY → DECIMAL CONVERSION

Step 1: Write the binary number. Step 2: Assign position numbers right to left from 0. Step 3: Multiply each bit by 2 raised to that position. Step 4: Add all results.

Example 1: Convert 11001 to Decimal

Bit:	1	1	0	0	1
Position:	4	3	2	1	0
Weight:	2^4	2^3	2^2	2^1	2^0
	= 16	8	4	2	1
Multiply:	1x16	1x8	0x4	0x2	1x1
=	16	+ 8	+ 0	+ 0	+ 1
	= 25 (Answer)				

□ Why is $0 \times 4 = 0$ and not 4?

The bit at position 2 is 0. Think of it like a light switch — bit=1 means ON (count the weight), bit=0 means OFF (weight is ignored). Zero multiplied by anything is always zero.

Example 2: Convert 10110 to Decimal

Bit:	1	0	1	1	0
Weight:	16	8	4	2	1
Multiply:	1x16	0x8	1x4	1x2	0x1
	= 16+0+4+2+0 = 22 (Answer)				

1.4 DECIMAL → BINARY CONVERSION

Repeatedly divide by 2. Write the remainder each time. When quotient reaches 0, stop. Read remainders from BOTTOM TO TOP — that is your binary answer.

Example: Convert 25 to Binary

25 / 2	= 12	remainder 1
12 / 2	= 6	remainder 0
6 / 2	= 3	remainder 0
3 / 2	= 1	remainder 1
1 / 2	= 0	remainder 1 STOP (quotient = 0)
Read bottom to top: 1 1 0 0 1 = 11001 (Answer)		

□ Common Confusion: 2 / 2 — Quotient vs Remainder

For 2 / 2: Quotient = 1 (2 fits into 2 exactly once) | Remainder = 0 (nothing left over).
Quotient = HOW MANY TIMES it fits. Remainder = WHAT IS LEFT OVER. Stop only when quotient = 0.

Example 2: Convert 19 to Binary

19/2=9	r1		9/2=4	r1		4/2=2	r0		2/2=1	r0		1/2=0	r1
Read bottom to top: 1 0 0 1 1 = 10011 (Answer)													

1.5 HEXADECIMAL → DECIMAL CONVERSION

Same method as binary to decimal but weights are powers of 16. Replace letters A-F with their decimal values before multiplying.

Example 1: Convert 1F to Decimal

Digit:	1	F
Position:	1	0

Weight: $16^1 16^0 = 16 \ 1$

F = 15 (from hex table)

Multiply: $1 \times 16 + 15 \times 1 = 16 + 15 = 31$ (Answer)

Example 2: Convert 2B to Decimal

B = 11 | $2 \times 16 + 11 \times 1 = 32 + 11 = 43$ (Answer)

Example 3: Convert 3F to Decimal

F = 15 | $3 \times 16 + 15 \times 1 = 48 + 15 = 63$ (Answer)

□ Where does 16 come from in the weight?

Hex is Base-16. So position 0 = $16^0 = 1$, position 1 = $16^1 = 16$, position 2 = $16^2 = 256$. Exactly the same logic as decimal uses powers of 10 and binary uses powers of 2.

1.6 OCTAL (BASE-8) KEY FACTS

- Digits: 0 to 7 only. Digits 8 and 9 do NOT exist in octal.
- 1 Octal digit = 3 Binary bits. Group binary in sets of 3 from right.
- 1 Hex digit = 4 Binary bits (called a nibble). Group binary in sets of 4 from right.

Binary to Octal shortcut:

Binary: 1 011 010 (group of 3 from right)

Groups: 1 3 2 = Octal: 132

1.7 2's COMPLEMENT — NEGATIVE NUMBERS IN COMPUTERS

Computers cannot store negative signs (+ or -). They use 2's complement to represent negative numbers using only 0s and 1s. The CPU converts subtraction into addition using this system internally.

Two steps — always in this exact order:

- STEP 1: Flip all bits (0 becomes 1, 1 becomes 0). This gives the 1's complement.
- STEP 2: Add 1 to the flipped result.

Example 1: 2's complement of 0101

Original: 0 1 0 1

Step 1 Flip: 1 0 1 0 (every bit flipped)

Step 2 Add1: 1 0 1 1 (Answer = 1011)

Example 2: 2's complement of 0011

Step 1 Flip: 0011 -> 1100

Step 2 Add1: 1100 + 1 = 1101 (Answer)

Example 3: 2's complement of 1111

Step 1 Flip: 1111 -> 0000

Step 2 Add1: 0000 + 1 = 0001 (Answer)

(1111 = -1 in decimal, its opposite +1 = 0001 — makes logical sense!)

Example 4: 2's complement of 0000 — Special Case

Step 1 Flip: 0000 → 1111

Step 2 Add1: 1111 + 1 = 10000 (5 bits appear)

Discard extra carry bit: 0000 (Answer)

□ Special Rule: Zero

Zero is the only number whose 2's complement is itself (0000). The extra 5th carry bit is discarded because we only have 4 bits.

1.8 QUICK CONVERSION SUMMARY

Conversion	Method	Key Rule
Binary to Decimal	Multiply each bit by 2^{position} , add all	0 bit = weight is ignored
Decimal to Binary	Divide by 2 repeatedly, read remainders bottom to top	Stop when quotient = 0
Hex to Decimal	Replace A-F with 10-15, multiply by 16^{position} , add	$16^1=16$, $16^2=256$
Binary to Octal	Group bits in 3s from right, convert each group	1 octal digit = 3 bits
Binary to Hex	Group bits in 4s from right, convert each group	1 hex digit = 4 bits

□ PRACTICE MCQs — With Answers

Q1. Binary 11111 in decimal?

- A) 30
- B) 31 ☒
- C) 32
- D) 63

→ $16+8+4+2+1 = 31$

Q2. Hex digit D equals which decimal?

- A) 12
- B) 14
- C) 13 ☒
- D) 15

→ A=10, B=11, C=12, D=13, E=14, F=15

Q3. Convert decimal 19 to binary.

- A) 10111
- B) 10011 ☒
- C) 11001
- D) 10001

→ Divide repeatedly — bottom to top gives 10011

Q4. 2's complement of 0011?

- A) 1100
- B) 1101 ☒

C) 1011

D) 1010

→ Flip-→1100, Add 1-→1101

Q5. One hex digit = how many bits?

A) 2

B) 3

C) 4 ☒

D) 8

→ 1 hex digit = 4 bits = 1 nibble

Q6. Hex 3F in decimal?

A) 53

B) 47

C) 63 ☒

D) 33

→ $3 \times 16 + 15 \times 1 = 48 + 15 = 63$

Q7. 2's complement of 0000?

A) 0000 ☒

B) 1111

C) 1000

D) 1001

→ Flip-→1111, Add1-→10000, discard carry-→0000

ROM = Read Only Memory. Stores permanent instructions like BIOS/firmware. Data survives power loss (non-volatile). Think of it as a printed book — easy to read, very hard to change.

Type	Full Form	Erased By	Times Writable	Used For
ROM	Read Only Memory	Cannot be erased	0 (factory only)	Fixed firmware
PROM	Programmable ROM	Cannot be erased	1 (user once)	Custom one-time firmware
EPROM	Erasable Programmable ROM	UV (Ultraviolet) light	Many	Dev/test — has glass window on chip
EEPROM	Electrically Erasable Programmable ROM	Electric signal (byte)	Many	BIOS settings storage
Flash	Flash Memory (advanced EEPROM)	Electric signal (block)	Many	Pen drives, SSD, SD cards

- ROM — Permanently written by manufacturer. End user cannot modify it at all.
- PROM — Shipped blank. User writes ONCE with a PROM burner. After that, permanent.
- EPROM — Has a visible glass window on chip. UV light shines through the window to erase.
- EEPROM — No window needed. Electrical signals erase individual bytes. Slower but precise.
- Flash — Evolved from EEPROM. Erases in blocks (not bytes) — much faster. Used in all modern storage.
- ALL ROM types are NON-VOLATILE — data survives when power is switched off.

Most Tested Pair: EEPROM vs Flash

Both use electrical signals to erase. The ONLY difference:

EEPROM erases byte by byte (slow, precise) | Flash erases block by block (fast, bulk).

Memory Trick

R=ROM fixed | P=PROM once | E=EPROM UV light | EE=EEPROM Electric byte | F=Flash Electric block

PRACTICE MCQs — With Answers

Q1. Which ROM is erased using UV light?

- A) PROM
- B) EEPROM
- C) EPROM
- D) Flash

→ EPROM has glass window for UV light exposure

Q2. Flash erases data in what unit?

- A) Bits
- B) Bytes
- C) Blocks ☒
- D) Pages

→ Flash=blocks, EEPROM=bytes — key distinction

Q3. PROM can be programmed how many times?

- A) Many
- B) Once ☒
- C) UV light
- D) Unlimited

→ PROM = write once then permanent

Q4. Which ROM is used in pen drives and SSDs?

- A) EPROM
- B) EEPROM
- C) PROM
- D) Flash ☒

→ Flash = fast block erase = modern storage

Q5. What is unique about EPROM's physical appearance?

- A) Blue
- B) Glass window ☒
- C) Larger
- D) Two pins

→ Glass window allows UV light to erase the chip

RAM = Random Access Memory. Temporary storage for currently running programs. VOLATILE — all data lost on power off. Think of it as your study table — only active work lives here.

Type	Storage Element	Speed	Cost	Needs Refresh?	Used In
SRAM	Flip-flops	Fastest	High	No	Cache memory (L1/L2/L3)
DRAM	Capacitors	Slow	Low	Yes — capacitors leak	Main RAM sticks
SDRAM	Capacitors	Faster	Low	Yes	Main RAM synced to CPU clock
DDR SDRAM	Capacitors	Fastest	Low	Yes	Modern RAM (DDR4/DDR5)

SRAM — Static RAM

- Uses flip-flops to store data. Flip-flops hold state without refreshing — that is why it is called STATIC.
- Very fast, very expensive. Used in CPU Cache (L1, L2, L3).

DRAM — Dynamic RAM

- Uses capacitors. Capacitors leak charge over time — data fades without refresh.
- Needs constant electrical refresh signal to maintain stored data.
- Slower but much cheaper. Your 8GB/16GB RAM stick is DRAM.

SDRAM — Synchronous DRAM

- Synchronised with CPU clock signal — reads/writes are timed exactly to CPU cycles. Faster than regular DRAM.

DDR SDRAM — Double Data Rate

- Transfers data TWICE per clock cycle — on both the rising and falling edge of the clock signal.
- DDR2 faster than DDR1, DDR3 faster than DDR2, DDR4 faster than DDR3. DDR5 is the current standard.

SRAM vs DRAM — Most Tested Pair

SRAM: Fast | Flip-flops | No refresh | Costly | Cache

DRAM: Slow | Capacitors | Needs refresh (capacitors leak!) | Cheap | Main RAM

Simple rule: More expensive = Faster = SRAM.

□ Memory Trick

Static=SRAM=no refresh=fast | Dynamic=DRAM=refresh needed=slow |

Double=DDR=twice per clock cycle

□ PRACTICE MCQs — With Answers

Q1. Which RAM is used in Cache Memory?

- A) DRAM
- B) SDRAM
- C) SRAM □
- D) DDR

→ SRAM=fast+expensive=cache

Q2. DRAM needs constant refreshing because it uses?

- A) Flip-flops
- B) Capacitors □
- C) Transistors
- D) Resistors

→ Capacitors leak charge — data fades without refresh

Q3. DDR SDRAM transfers data how many times per clock?

- A) Once
- B) Twice □
- C) Four
- D) Eight

→ Double Data Rate = 2 transfers per cycle

Q4. Which is faster — SRAM or DRAM?

- A) DRAM
- B) Both same
- C) SRAM □
- D) Depends

→ SRAM=flip-flops=no refresh=faster

Q5. SDRAM is faster than DRAM because?

- A) Flip-flops
- B) No refresh
- C) Synced with CPU clock □
- D) Cheaper

→ Synchronous = timed to CPU clock = no waiting

Memory is arranged in levels. Closer to CPU = faster, smaller, more expensive. Further from CPU = slower, larger, cheaper. The hierarchy exists because fast memory is very expensive — you cannot make all memory as fast as registers.

Level	Memory	Speed	Size	Cost	Volatile ?	Location
1	Registers	Fastest	Bytes	Highest	Yes	Inside CPU
2	Cache (L1/L2/L3)	Very Fast	KB-MB	High	Yes	On/near CPU chip
3	Main Memory (RAM)	Medium	GB	Medium	Yes	Motherboard slots
4	Secondary Storage	Slowest	TB	Lowest	No	HDD, SSD, USB

- Speed order (fastest first): Registers > Cache > RAM > Secondary
- Cost order (costliest first): Registers > Cache > RAM > Secondary
- Size order (largest first): Secondary > RAM > Cache > Registers
- Cache reduces CPU's need to access slower RAM — stores frequently used data.
- Registers are inside the CPU — hold data being actively processed right now.
- All levels except Secondary are VOLATILE. Secondary (HDD/SSD) is NON-VOLATILE.

Memory Trick

Rich Cache Rarely Sleeps -> R=Registers | C=Cache | R=RAM | S=Secondary Going left to right: Speed decreases, Size increases, Cost decreases.

PRACTICE MCQs — With Answers

Q1. Fastest memory in the hierarchy?

- A) Cache
- B) RAM
- C) Registers
- D) SSD

→ Registers inside CPU = fastest

Q2. Cache sits between which two levels?

- A) RAM and HDD
- B) CPU and RAM
- C) Registers and CPU
- D) ROM and RAM

→ Cache buffers fast CPU and slower RAM

Q3. Largest memory in size?

- A) Cache

- B) Registers
- C) RAM
- D) Secondary ☒

→ HDD/SSD = terabytes

Q4. Which level is NON-VOLATILE?

- A) Registers
- B) Cache
- C) RAM
- D) Secondary ☒

→ Secondary storage retains data without power

Q5. Correct speed order?

- A) RAM>Cache>Reg>Sec
- B) Reg>Cache>RAM>Sec ☒
- C) Cache>Reg>RAM>Sec
- D) Sec>RAM>Cache>Reg

→ Registers > Cache > RAM > Secondary

CPU = Central Processing Unit = brain of the computer. Every instruction passes through it. Three main parts: ALU (does the work), CU (gives orders), Registers (hold temporary data).

ALU — Arithmetic Logic Unit

Performs ALL arithmetic (add, subtract, multiply, divide) and ALL logical operations (AND, OR, NOT, comparisons like greater-than, equal-to). Every single calculation happens here.

CU — Control Unit

Directs all parts of the computer. Tells ALU when to compute, memory when to read/write, coordinates I/O. Does NOT process data itself — it only controls and coordinates.

Registers

Ultra-fast storage locations inside the CPU. Hold data being processed right NOW. Several specialised registers each with a specific job.

Register	Full Form	What It Holds
PC	Program Counter	Address of the NEXT instruction. Auto-increments after each fetch.
ACC	Accumulator	Result of the most recent ALU operation.
MAR	Memory Address Register	Address of the memory location being accessed.
MDR	Memory Data Register	Actual data being transferred to or from memory.
IR	Instruction Register	The CURRENT instruction being decoded and executed.
SP	Stack Pointer	Address of the top of the current stack in memory.

Fetch-Decode-Execute Cycle:

1. FETCH	PC holds address of next instruction. CPU reads it from memory into IR. PC then increments.
2. DECODE	CU reads the instruction in IR and determines what operation is needed and what data is required.
3. EXECUTE	ALU or relevant unit carries out the instruction. Result stored in ACC or written back to memory.

□ Key Fact About CU

CU controls everything but performs NO computation itself. ALU does ALL actual computation. This is a very common trap question in C-CAT.

□ **PRACTICE MCQs — With Answers**

Q1. Which performs arithmetic and logical operations?

- A) CU
- B) MDR
- C) ALU □
- D) PC

→ ALU = Arithmetic Logic Unit = all math and logic

Q2. Program Counter holds?

- A) Current result
- B) Address of next instruction □
- C) Memory data
- D) ALU output

→ PC always points to the NEXT instruction

Q3. Which unit controls all other units?

- A) ALU
- B) Register
- C) MDR
- D) CU □

→ CU = Control Unit = the manager, does no computation

Q4. MDR holds?

- A) Next instruction address
- B) Stack top
- C) Actual data transferred □
- D) Current instruction

→ MDR=data value, MAR=address — don't confuse them

Q5. Correct CPU cycle order?

- A) Execute-Fetch-Decode
- B) Decode-Fetch-Execute
- C) Fetch-Decode-Execute □
- D) Fetch-Execute-Decode

→ Always: Fetch then Decode then Execute

Algorithm = step-by-step written solution in words. Flowchart = same solution drawn with symbols. Algorithm = recipe in words. Flowchart = recipe in pictures. Same logic, different representation.

5 Properties of a Valid Algorithm — All 5 Required

Property	Meaning	Exam Trap?
Input	Zero or more inputs given to the algorithm	Trick: zero inputs is valid!
Output	At least one output must be produced	Must produce a result
Definiteness	Every step is clear and unambiguous	No step can mean two things
Finiteness	Must terminate after finite number of steps	INFINITY is NOT a property — trap!
Effectiveness	Each step is basic and feasible to carry out	Steps must be executable

Flowchart Symbols

Symbol Shape	Name	Used For	Example
Oval	Terminal (Start/Stop)	Beginning and end of flowchart	START, STOP, END
Rectangle	Process	Calculations and assignments	Sum = A + B
Diamond	Decision	Yes/No branching condition	Is X > 5?
Parallelogram	Input / Output	Read input or display output	READ N, PRINT Result
Arrow	Flow Line	Direction between steps	Connects all symbols

- Exactly ONE Start symbol and ONE Stop symbol in every flowchart.
- Diamond always has exactly TWO exits — YES and NO. Never one, never three.
- Loop = an arrow pointing BACK to a previous step + decision diamond to exit.

Memory Trick

Old Professors Draw Pretty Arrows -> O=Oval | P=Process(Rect) | D=Diamond |
P=Parallelogram | A=Arrows

Q1. Decision symbol?

- A) Rectangle
- B) Oval
- C) Diamond ☒
- D) Parallelogram

→ Diamond=Decision=YES and NO paths

Q2. Start/Stop symbol?

- A) Diamond
- B) Rectangle
- C) Parallelogram
- D) Oval ☒

→ Oval=Terminal=Start/Stop

Q3. Input/Output symbol?

- A) Oval
- B) Rectangle
- C) Parallelogram ☒
- D) Diamond

→ Parallelogram=I/O, Rectangle=Process

Q4. NOT a property of algorithm?

- A) Finiteness
- B) Effectiveness
- C) Infinity ☒
- D) Definiteness

→ Algorithm MUST end. Infinity is never a property.

Q5. Flowchart must have how many Start symbols?

- A) Two
- B) As many as needed
- C) One ☒
- D) None

→ Exactly ONE Start and ONE Stop always

Registers are the fastest, smallest, most expensive memory. Located directly inside the CPU. Hold data being processed right now.

- Fastest and most expensive memory in the entire hierarchy.
- Located INSIDE the CPU — not separate like RAM or Cache.
- Size: 8 to 64 bits typically. Part of Fetch-Decode-Execute cycle.

Register	Full Form	Holds
PC	Program Counter	Address of NEXT instruction — auto-increments after fetch
ACC	Accumulator	Result of ALU operation
MAR	Memory Address Register	Memory address to read/write
MDR	Memory Data Register	Actual data being transferred to/from memory
IR	Instruction Register	Current instruction being executed
SP	Stack Pointer	Address of top of the stack

▣ PRACTICE MCQs — With Answers

Q1. Which holds address of next instruction?

- A) ACC
- B) MAR
- C) PC ☒
- D) MDR

→ PC=Program Counter=next instruction address

Q2. IR stands for?

- A) Internal Register
- B) Instruction Register ☒
- C) Index Register
- D) Input Register

→ IR=Instruction Register=current instruction

Q3. Which holds the ALU result?

- A) PC
- B) SP
- C) MAR
- D) ACC ☒

→ ACC=Accumulator=stores ALU output

Cache is small, super-fast memory between CPU and RAM. CPU checks cache first. If found = fast (cache hit). If not found = go to slow RAM (cache miss).

- Cache uses SRAM — flip-flops, no refresh, fastest after registers.
- L1 Cache — inside CPU chip. Smallest, fastest. Size: KB range.
- L2 Cache — near CPU. Bigger than L1, slightly slower.
- L3 Cache — shared across all CPU cores. Largest cache, slowest of three.
- Cache Hit — data found in cache = fast access.
- Cache Miss — data NOT in cache = must go to RAM = slow.
- Hit Ratio = Cache Hits / Total Accesses. Higher is better.

Memory Trick

L1=Fastest Smallest Inside CPU | L2=Bigger Slower Near CPU | L3=Largest Shared
Hit=Found=Fast | Miss=Not Found=RAM=Slow

PRACTICE MCQs — With Answers

Q1. Which cache level is inside the CPU?

- A) L3
- B) L2
- C) L1 ☒
- D) L4

→ L1=inside CPU chip=fastest

Q2. Cache Hit means?

- A) Data in RAM
- B) Data found in Cache ☒
- C) Cache full
- D) Cache missed

→ Hit=found in cache=fast access

Q3. Hit Ratio formula?

- A) Misses/Total
- B) Hits/Misses
- C) Hits/Total ☒
- D) Total/Hits

→ Hit Ratio = Cache Hits / Total Accesses

Q4. L1, L2, L3 — which is largest?

- A) L1
- B) L2
- C) L3 ☒
- D) Equal

→ L3=shared across cores=largest cache

When CPU communicates with devices (keyboard, printer, disk), it uses IO techniques. The key question: who controls the transfer and how?

Technique	How It Works	CPU Involvement	Efficiency
Programmed IO	CPU continuously polls device (busy-waiting)	100% — CPU stuck waiting	Worst
Interrupt-Driven	Device signals CPU when ready via interrupt	Partial — handles interrupt only	Better
DMA	DMA Controller transfers data directly to memory	Minimal — start and end only	Best

- Programmed IO — CPU wastes time polling. Also called busy-waiting.
- Interrupt-Driven IO — Device fires interrupt, CPU runs ISR (Interrupt Service Routine).
- DMA — DMA Controller is separate hardware chip managing bulk data transfers.
- ISR = Interrupt Service Routine — the code that runs when interrupt fires.
- Efficiency order: Programmed IO < Interrupt IO < DMA.

Memory Trick

Programmed=Wait at counter | Interrupt=Get notification | DMA=Home delivery
PIO < Interrupt < DMA

PRACTICE MCQs — With Answers

Q1. Which IO wastes most CPU time?

- A) DMA
- B) Interrupt
- C) Programmed IO
- D) ISR

→ Programmed IO=CPU busy-waiting=most wasteful

Q2. In DMA, who transfers data?

- A) CPU
- B) ALU
- C) DMA Controller
- D) CU

→ DMA Controller handles transfer directly

Q3. ISR stands for?

- A) Interrupt Signal Register
- B) Interrupt Service Routine
- C) IO Service Register
- D) Interrupt System Routine

→ ISR=code that runs when interrupt fires

Q4. Correct efficiency order?

A) DMA>Interrupt>PIO ☒

B) PIO>DMA>Interrupt

C) Interrupt>DMA>PIO

D) All equal

→ DMA most efficient, PIO least efficient

Languages are classified by closeness to machine hardware. Low level = close to machine. High level = close to human. Like speaking directly to a robot (low) vs speaking English (high).

Language	Level	Translator Needed	Speed	Portable?	Example
Machine Language	Lowest	None — CPU reads directly	Fastest	No	0s and 1s
Assembly Language	Low	Assembler	Fast	No	MOV, ADD, SUB
High Level Language	High	Compiler or Interpreter	Slower	Yes	Python, C, Java

- Assembler — translates Assembly to Machine code.
- Compiler — translates ENTIRE program at once. Faster execution. (C, C++)
- Interpreter — translates LINE BY LINE. Slower but easier to debug. (Python)
- High Level Languages score high on readability and portability.
- Low Level Languages score high on efficiency (close to hardware).

Memory Trick

Low Level=Fast+Hardware Dependent | High Level=Slow+Portable
 Compiler=Whole Program at Once | Interpreter=Line by Line (Python)

PRACTICE MCQs — With Answers

Q1. Which language is directly understood by CPU?

- A) Assembly
- B) Python
- C) Machine Language ☒
- D) C++

→ Machine language=binary=CPU reads directly

Q2. Assembler converts which language?

- A) Python
- B) Assembly ☒
- C) C++
- D) Java

→ Assembler=Assembly language translator

Q3. Which translates line by line?

- A) Compiler
- B) Both
- C) Neither

D) Interpreter ☒

→ Interpreter=line by line=Python uses it

Q4. Python uses which translator?

A) Assembler

B) Compiler

C) Interpreter ☒

D) Linker

→ Python=Interpreter=line by line

These are the standard criteria used to judge how good or suitable a programming language is for a given purpose.

Criteria	Meaning	HLL or LLL Scores Higher?
Readability	How easily code can be read and understood	HLL (English-like)
Writability	How easily code can be written	HLL (simpler syntax)
Reliability	Consistent correct performance, good error handling	HLL (safer)
Portability	Runs on different machines/OS without changes	HLL (platform independent)
Efficiency	Fast execution with minimal memory usage	LLL (close to hardware)
Maintainability	How easily code can be updated or fixed	HLL (more readable)
Simplicity	Fewer features = easier to learn and use	HLL (usually)

Memory Trick

Read Write Rely Roam Race -> Readability | Writability | Reliability | Portability | Efficiency
HLL wins all EXCEPT Efficiency where LLL wins.

PRACTICE MCQs — With Answers

Q1. Which checks if code runs on different machines?

- A) Readability
- B) Reliability
- C) Portability ☒
- D) Writability

→ Portability=runs on different platforms

Q2. Low level languages score highest on?

- A) Readability
- B) Portability
- C) Efficiency ☒
- D) Maintainability

→ LLL=close to hardware=most efficient

Q3. Reliability means?

- A) Easy to write
- B) Runs on all OS

C) Consistent correct performance □

D) Easy to read

→ Reliability=consistent+correct+handles errors

Object-Oriented Programming models real-world things as objects. Class=blueprint. Object=instance. OOP languages include Python, Java, C++.

Concept	Definition	Real-Life Analogy
Class	Blueprint/template for creating objects	Blueprint of a house
Object	Instance of a class	An actual built house
Encapsulation	Bundle data + methods together, hide internal details	A capsule pill (medicine inside)
Inheritance	Child class gets properties of parent class	Child inherits traits from parents
Polymorphism	Same method name, different behaviour based on context	A person acts differently at work vs home
Abstraction	Hide complexity, show only essential details	Car accelerator — you don't see engine
Constructor	Special method that runs automatically on object creation	Auto-runs at object birth

□ Abstraction vs Encapsulation — Most Confused Pair

Abstraction = HIDE COMPLEXITY (show only what is needed).

Encapsulation = BUNDLE data and methods TOGETHER.

Trick: Abstract=hide details. Encapsulate=bundle together in a capsule.

□ PRACTICE MCQs — With Answers

Q1. A class is best described as?

- A) Object
- B) Function
- C) Blueprint □
- D) Variable

→ Class=blueprint, Object=instance

Q2. Child class uses parent properties — which concept?

- A) Polymorphism
- B) Encapsulation
- C) Abstraction
- D) Inheritance □

→ Inheritance=child gets parent properties

Q3. Hiding complexity, showing only essentials?

- A) Encapsulation
- B) Abstraction ☒
- C) Polymorphism
- D) Inheritance

→ Abstraction=hide complexity

Q4. Same method name, different behaviour?

- A) Abstraction
- B) Inheritance
- C) Polymorphism ☒
- D) Encapsulation

→ Polymorphism=many forms, same name

Q5. Bundling data and methods and hiding internals?

- A) Abstraction
- B) Inheritance
- C) Polymorphism
- D) Encapsulation ☒

→ Encapsulation=bundle+hide internals

Data Types define what kind of value a variable holds. Data Structures define how data is organised in memory. Type = type of box. Structure = how boxes are arranged.

Type	Example	Holds
int	1, 100, -5	Whole numbers
float	3.14, 2.5	Decimal numbers
char	'A', 'z'	Single character
boolean	True, False	Only two values
String	'Hello'	Sequence of characters
Array	[1,2,3]	Collection of same type

Structure	Principle	Add	Remove	Used For
Array	Fixed size, index from 0	Direct index	Direct index	Sequential data
Stack	LIFO — Last In First Out	Push	Pop	Undo, recursion
Queue	FIFO — First In First Out	Enqueue	Dequeue	Scheduling, printing
Linked List	Nodes connected by pointers	Dynamic	Dynamic	Dynamic size data
Tree	Hierarchical — root at top	Insert	Delete	File systems, databases

□ Memory Trick

Stack=LIFO=Pile of plates (last placed=first removed) Queue=FIFO=Ticket counter (first in=first served)

□ PRACTICE MCQs — With Answers

Q1. Stack follows which principle?

- A) FIFO
- B) FILO
- C) LIFO □
- D) LILO

→ Stack=LIFO=Last In First Out

Q2. Array index starts at?

- A) 1

B) -1

C) 0 ☒

D) Depends

→ Arrays always 0-indexed

Q3. Queue operation for adding?

A) Push

B) Pop

C) Enqueue ☒

D) Dequeue

→ Queue: Enqueue=add, Dequeue=remove

Q4. Which uses pointers to connect nodes?

A) Array

B) Stack

C) Queue

D) Linked List ☒

→ Linked List=nodes connected by pointers

Variables are named memory locations that store values. Syntax = grammar rules of a programming language. Violate syntax = program will not compile.

- Variable — named memory location. Value CAN change during execution.
- Constant — value CANNOT change once assigned. Example: $\pi = 3.14$
- Declaration — naming variable and type. Example: `int x;`
- Initialisation — assigning value at declaration. Example: `int x = 5;`
- Keywords — reserved words (`int`, `if`, `while`, `return`) — CANNOT be used as variable names.
- Syntax Error — code breaks grammar rules, program will not run.

Valid Identifier Rules:

OK `my_marks` -> underscore is allowed as special character

BAD `2marks` -> cannot START with a number

BAD `my marks` -> no spaces allowed

BAD `my-marks` -> hyphens not allowed (only underscore)

BAD `int` -> keyword, cannot use as variable name

Memory Trick

Valid = starts with letter or underscore | only letters+digits+underscore after | no spaces no hyphens no keywords

PRACTICE MCQs — With Answers

Q1. Which is a valid variable name?

- A) `2marks`
- B) `my marks`
- C) `my_marks` ☒
- D) `my-marks`

→ Only underscore allowed as special character

Q2. Declaration vs Initialisation?

- A) Same thing
- B) Declaration assigns value
- C) Declaration=name+type, Init=assign value ☒
- D) None

→ `int x;` =declaration | `int x=5;` =initialisation

Q3. Keywords as variable names?

- A) Yes always
- B) Only Python
- C) No never ☒
- D) Only C++

→ Keywords are reserved — never usable as variable names

Q4. `int x;` is an example of?

- A) Initialisation
- B) Declaration ☒
- C) Assignment
- D) Keyword

→ `int x;` declares type and name only — no value assigned

Flow control decides the ORDER in which statements execute. Without it every program just runs top to bottom. Flow control adds decisions, loops, and jumps.

Structure	Type	When to Use
if / if-else	Selection	Execute based on a condition
switch	Selection	Multiple fixed choices
for loop	Iteration	Number of repetitions KNOWN
while loop	Iteration	Repeat WHILE condition is true — count unknown
do-while	Iteration	Must execute AT LEAST ONCE — check condition after

- break — exits the loop immediately and goes to next statement after loop.
- continue — skips current iteration only and jumps to next iteration.

□ while vs do-while

while — checks condition FIRST. If false from start, body NEVER executes.

do-while — executes body FIRST, then checks condition. Always runs at LEAST ONCE.

□ Memory Trick

for=count KNOWN | while=condition unknown | do-while=at least once break=EXIT loop | continue=SKIP this round

□ PRACTICE MCQs — With Answers

Q1. Which loop executes at least once?

- A) for
- B) while
- C) do-while □
- D) None

→ do-while checks condition AFTER first execution

Q2. break statement does?

- A) Skip iteration
- B) Exit loop completely □
- C) Restart loop
- D) End program

→ break=immediately exits the entire loop

Q3. Selection structure uses?

- A) Iteration
- B) Sequence
- C) if-else and switch □

D) Jump

→ Selection=if/else/switch=decision making

Q4. continue does?

A) Exit loop

B) End program

C) Skip current iteration ☒

D) Restart loop

→ continue=skip this round, go to next iteration

Q5. Best loop when count is known?

A) while

B) do-while

C) for ☒

D) if-else

→ for loop=designed for known number of iterations

- IDE = Integrated Development Environment. All-in-one: editor + compiler + debugger. Examples: VS Code, Eclipse, IntelliJ.
- Debugging = finding and fixing errors. Three types: Syntax Error (grammar — won't run), Runtime Error (crashes while running), Logical Error (wrong output but still runs).
- Breakpoint = marker that pauses program at that line so you can inspect variable values.

□ PRACTICE MCQs — With Answers

Q1. Full form of IDE?

- A) Internal Dev Env
- B) Integrated Development Environment □
- C) Integrated Debug Engine
- D) Internal Debug Engine

→ IDE=Integrated Development Environment

Q2. Which error gives wrong output but program still runs?

- A) Syntax
- B) Runtime
- C) Logical □
- D) Compile

→ Logical error=wrong output, no crash

Q3. Breakpoint is used to?

- A) End program
- B) Skip line
- C) Pause execution at a specific line □
- D) Compile faster

→ Breakpoint=pause here to inspect values

- Functional Programming = style where functions are main building blocks. No changing variables. Uses recursion instead of loops. Examples: Haskell, Lisp.
- Pure Function = always gives same output for same input. No side effects.
- Immutability = variables cannot be changed once assigned.

□ PRACTICE MCQs — With Answers

Q1. Variables cannot change once assigned — this is?

- A) Inheritance
- B) Immutability □
- C) Polymorphism
- D) Encapsulation

→ Immutability=cannot change after assignment

Q2. Pure function means?

- A) Written in Python
- B) Uses loops
- C) Same input always gives same output □
- D) No parameters

→ Pure function=deterministic=no side effects

MASTER QUICK REFERENCE — ALL 17 TOPICS

Powers of 2 and 16

n	2 ⁿ (Binary)	16 ⁿ (Hex)
0	1	1
1	2	16
2	4	256
3	8	4096
4	16	65536
5	32	—
6	64	—
7	128	—

ROM Types

Type	Erase Method	Times Writable
ROM	Cannot erase	0 (factory only)
PROM	Cannot erase	1 (user once)
EPROM	UV light	Many
EEPROM	Electric signal (byte by byte)	Many
Flash	Electric signal (block by block)	Many

RAM Types

Type	Element	Speed	Used In
SRAM	Flip-flops	Fastest	Cache
DRAM	Capacitors	Slow	Main RAM
SDRAM	Capacitors	Faster	Main RAM
DDR	Capacitors	2x per cycle	Modern RAM

Memory Hierarchy

Level	Memory	Speed	Volatile?
1	Registers	Fastest	Yes
2	Cache	Very Fast	Yes
3	RAM	Medium	Yes
4	Secondary	Slowest	No

CPU Registers

Register	Holds
PC	Address of NEXT instruction
ACC	ALU result
MAR	Memory address to access
MDR	Actual data transferred
IR	Current instruction
SP	Top of stack

Flowchart Symbols

Shape	Represents
Oval	Start / Stop
Rectangle	Process (calculation)
Diamond	Decision (Yes/No)
Parallelogram	Input / Output
Arrow	Flow direction

OOP Four Pillars

Pillar	One-Line Definition
Encapsulation	Bundle data+methods, hide internals
Inheritance	Child class gets parent class properties
Polymorphism	Same method name, different behaviour
Abstraction	Hide complexity, show only essentials